

**Yenepoya Institute of Arts, Science,
Commerce, and Management**

Project – Low Level Design

on

Big Data Analytics for Retail Demand

Student Name: HARINAND.A

Industry Mentor:

SHASHANK KORRAM
SHAROLPRABHAKAR

Guided by:

SUZANA

Table of Contents

No.	Section Title	Page
1	Introduction	
1.1	Scope of the Document	
1.2	Intended Audience	
1.3	System Overview	
2	System Design	
2.1	Application Design	
2.2	Process Flow	
2.3	Information Flow	
2.4	Components Design	
2.5	Key Design Considerations	
2.6	API Catalogue	
3	Data Design	
3.1	Data Model	
3.2	Data Access Mechanism	
3..3	Data Retention Policies	
3.4	Data Migration	
4	Interfaces	
5	State and Session Management	
6	Caching	
7	Non-Functional Requirements	
7.1	Security Aspects	
7.2	Performance Aspects	
8	References	

1. Introduction

The Low-Level Design (LLD) of the Retail Demand Analysis system provides a detailed technical description of the internal working of the system. It explains how each module is implemented using Python, Pandas, and SQLite.

Unlike the High-Level Design, which focuses on overall architecture, this document describes the exact logic, functions, data transformations, and processing steps used to analyze retail data. The system processes transactional data through multiple stages including data loading, cleaning, transformation, aggregation, storage, and visualization.

1.1 Scope of the Document

This document focuses on the detailed implementation of the system.

In Scope:

- Data loading and preprocessing
- Data transformation and feature engineering
- SQL query execution
- Visualization and insights

Out of Scope:

- High-level architecture design
- External system integration
- Real-time systems

1.2 Intended Audience

- Developers
- Project Evaluators
- Students learning data analytics

1.3 System Overview

The system operates as a step-by-step data processing pipeline.

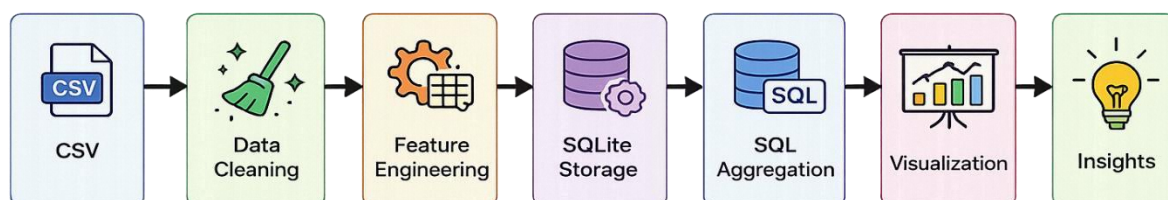
Data is loaded from a CSV file, cleaned, transformed, and aggregated using Pandas. The processed data is then stored in a SQLite database, and SQL queries are used to retrieve results for visualization.

The system follows an ETL pipeline:

- Extract: Data loaded from CSV
 - Transform: Data cleaned and transformed using Pandas
 - Load: Processed data stored into SQLite for SQL-based analysis
2. System Design

2.1 Application Design

The system is implemented using a modular pipeline architecture. Each module performs a specific task and passes the processed data to the next stage.



2.2 Process Flow

1. Load dataset from CSV file
2. Perform data cleaning (handle missing values, convert date format)
3. Apply feature engineering (Month, Revenue in Millions, DayOfWeek)
4. Store processed data into SQLite database
5. Execute SQL queries for aggregation (GROUP BY, SUM)
6. Generate visualizations (bar charts, line charts, pie charts)
- 7.

Derive insights and conclusions

2.3 Information Flow

Input:

- CSV dataset

Processing:

- Data cleaning (handling missing values, formatting)
- Feature engineering (Month, DayOfWeek, Revenue)
- SQL-based aggregation (GROUP BY, SUM)
- Data retrieval using SQL queries

Output:

- Tables
- Graphs
- Insights

2.4 Components Design

Data Loading Module

```
df = pd.read_csv("Retail_Transaction_Dataset.csv")  
df.head()
```

Data Cleaning Module

```
df.dropna(inplace=True)  
  
df["TransactionDate"] = pd.to_datetime(df["TransactionDate"])  
  
df.drop_duplicates(inplace=True)
```

Feature Engineering Module

```
df["Month"] = df["TransactionDate"].dt.month  
  
df["DayOfWeek"] = df["TransactionDate"].dt.day_name()  
  
df["Revenue"] = df["Quantity"] * df["Price"]
```

Aggregation Module

```
query = """  
SELECT ProductCategory, SUM(Revenue) AS TotalRevenue  
FROM sales  
GROUP BY ProductCategory  
ORDER BY TotalRevenue DESC  
"""  
  
result = pd.read_sql(query, conn)
```

Database Module

```
conn = sqlite3.connect("retail.db")  
  
df.to_sql("sales", conn, if_exists="replace", index=False)
```

Visualization Module

```
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(8,5))  
  
plt.bar(result["ProductCategory"], result["TotalRevenue"])  
  
plt.title("Revenue by Product Category")  
plt.xlabel("Category")  
plt.ylabel("Revenue")  
  
plt.xticks(rotation=45)  
  
plt.show()
```

2.5 Key Design Considerations

- Efficient data processing
- Accuracy of transformation
- Maintainability of code
- Simplicity of implementation

2.6 API Catalogue

No APIs are used. SQL queries act as the data retrieval mechanism.

3. Data Design

3.1 Data Model

CustomerID	INTEGER	Unique identifier for each customer
ProductID	TEXT	Identifier for product (A, B, C, D)
Quantity	INTEGER	Number of units purchased
Price	REAL	Price per unit
TransactionDate	DATETIME	Date and time of transaction
PaymentMethod	TEXT	Mode of payment (Cash, Credit Card, Debit Card, P
ProductCategory	TEXT	Category of product (Books, Electronics, Clothing, Home Decor)
DiscountApplied (%)	REAL	Discount percentage applied
TotalAmount	REAL	Final amount after discount
Month	INTEGER	Extracted month from TransactionDate
DayOfWeek	TEXT	Day name for weekly trend analysis
Revenue	REAL	Total Revenue (same as TotalAmount)

3.2 Data Access Mechanism

- Pandas → Data cleaning, transformation, and feature engineering
- SQLite → Data storage
- SQL → Aggregation and analytical queries (GROUP BY, SUM)
- Pandas → Data retrieval from SQL for visualization

3.3 Data Retention Policies

- Data stored locally in SQLite database
- No automatic deletion; data retained for analysis purposes
- Dataset used for academic/analysis purposes only

3.4 Data Migration

Not applicable

4. Interfaces

- CSV File → Input data source loaded into Pandas DataFrame
- Pandas ↔ SQLite → Data is stored and retrieved using SQLite connection
- SQL Queries → Used for aggregation and data analysis
- Jupyter Notebook → Execution environment for data processing and visualization

5. State and Session Management

- Data is loaded into memory as a Pandas DataFrame during execution
- Intermediate transformations are handled within the DataFrame
- Final processed data is stored in SQLite database
- No session persistence beyond execution (stateless processing)

6. Caching

- Not implemented due to small dataset size
- Data processing is efficient and does not require caching mechanisms

7. Non-Functional Requirements

7.1 Security Aspects

- Local execution environment (no cloud exposure)
 - No external data sharing or API integration
 - Data stored securely in local SQLite database
-

7.2 Performance Aspects

The system is optimized for performance within a local environment:

- Uses Pandas vectorized operations for efficient data processing
- Processes a manageable dataset efficiently (simulating Big Data concepts)
- SQLite enables fast query execution for aggregated results
- SQL aggregation reduces data size before visualization, improving performance
- Visualization is generated quickly using summarized data

The system performs batch processing and does not support realtime streaming.

8. References

1. Pandas Documentation : <https://pandas.pydata.org/docs/>
2. Python Documentation : <https://docs.python.org/3/>
3. Dataset (Kaggle) :
<https://www.kaggle.com/datasets/fahadrehman07/retail-transaction-dataset>